# Autonomous Systems and Robotics 700 Assignment

RICHFIELD
richfield.ac.za

**Student ITS No:** 402311859

**Qualification:** BSc: IT **Year of Study:** 3rd **Semester:** 2nd

**Assignment due date:** 23 October 2026 **Date submitted:** 12 October 2026

| QUESTION | EXAMINER MARKS | MODERATOR MARKS | REMARKS |
|---|---|---|---|
| COMPONENT A | | | |
| COMPONENT B | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

**ASSIGNMENT INSTRUCTIONS**

***Please tick each box to confirm completion.***

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

**For essay-style assignments, please include the following sections:**

Table of

Contents

Introduction

Main Body (with relevant

subheadings) Conclusion

References

Submit the assignment in PDF

format on Moodle. Use the

specified cover page provided.

Include a signed declaration of originality.

**DECLARATION OF ORIGINALITY:**

I hereby declare that this assignment is my own work and has not been copied from any other source except where due

acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

**Student Signature:** **Date:**

LESEDI MAGAKWE 12 OCTOBER 2026

**Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.**

# Table of Contents

**Autonomous Systems and Robotics 700 Assignment** ..... 1
1. Introduction ..... 4
2. Methodology ..... 5
2.1 Component A: Perception, Planning, and Control ..... 5
2.2 Component B: Multi-Agent Swarm with Learning ..... 7
3. Results and Evaluation ..... 9
3.1 Component A Results ..... 9
3.2 Component B Results ..... 11
4. Ethics, Risk and Resposible Autonomy ..... 15
4.1 Introduction ..... 15
4.2 Bias and Reliability in Sensors, Simulation and Learned Models ..... 15
4.3 Transparency and Explainability ..... 15
4.4 Autonomy in Safety-Critical and Human-Shared Environments ..... 16
4.5 Fault Tolerance and Failure-Mode Mitigation in Multi-Agent Systems ..... 17
4.6 Responsible Deployment in the South African Context ..... 17
4.7 Conclusion ..... 18
References ..... 19

# 1. Introduction

This report describes the design, development, and evaluation of two intelligent robotic systems which have been created using Webots simulation software and Python programming language coupled with Matplotlib for visualization and analysis. This project shows how sensor perception, motion planning, control theory, multi-agent systems, and machine learning can be implemented to create full-fledged autonomous systems, and how their ethical ramifications can be analyzed.

In the first part of the project, Component A deals with the basic pipeline of a single autonomous robot, where sensor data is simulated, an environment model is created, an optimal path is calculated using classical graph searching algorithm and is executed using closed-loop control mechanism called Proportional-Integral-Derivative control. In the second part of the project, Component B, multiple robots are coordinated in a swarm to reach common consensus about the system state using average consensus algorithm, PSO algorithm for task allocation and decision making is done by a neural network.

The development purposely excludes ROS and Gazebo from use, relying solely on Webots and python packages (Matplotlib, NumPy, scikit-learn). None of the used algorithms are obtained from any pre-made packages, which makes possible to observe and assess the inner computational behavior of these algorithms.

One of the important themes of the report is the difference between how well autonomous systems show their performance in the simulations and how well they perform in reality. This topic is discussed in great detail in the Ethical Section of the report.

# 2. Methodology

## 2.1 Component A: Perception, Planning, and Control

**Simulation environment**. A 5x5 m simulation environment was designed in Webots using a differential drive e-puck robot and four geometric shapes (cone, cylinder, sphere, and capsule) positioned at (-2, 0), (0, 1), (2, 0), and (0, -2). The world file (component_a.wbt) and controller code (my_controller.py) are located in worlds/ and controllers/ directories of the project respectively.

**Perception Pipeline.** The eight proximity sensors were sampled every 32 ms (simulation step) and augmented by the occupancy grid map of size 0.1 m x 0.1 m where each cell is considered to be occupied if the centre of the cell lies inside the circle of known obstacle or the boundary of the arena. This occupancy grid is generated programmatically by occupancy_grid.py code and served as an input for both motion planners. The reason for selecting occupancy grid rather than sensor threshold values was to enable algorithm comparison while ensuring both planners work on an identical and deterministic perception representation.

**Motion Planning.** Two shortest-path algorithms have been implemented from scratch:

**Dijkstra's Algorithm** – uses priority queue with total cost from start node as the key for each node. Explorations happen in all directions until the goal state is found. Is an optimal algorithm.

**A* Algorithm** – uses f(n) = g(n) + h(n) where g(n) is total cost from start state and h(n) is Euclidean heuristic function from current node to goal state. Is an optimal algorithm if h(n) is admissible.

Both models utilized 8-connected neighborhood with cost value of 1.0 for straight movements and cost value of $\sqrt{2}$ for diagonal movements. The comparison of the two algorithms was carried out based on the path length, number of explored nodes, and time spent for the computations.

Dijkstra vs A* - Comparison

| Metric | Dijkstra | A* |
|---|---|---|
| Path length (cells) | 56.57 | 56.57 |
| Nodes explored | 2135 | 41 |
| Time (ms) | 11.374 | 0.571 |

*Figure 1: Occupancy grid representation of the arena.*

**Control system.** The kinematic model of the differential drive robot was developed based on:

$v = (v_R + v_L) / 2$

$\omega = (v_R - v_L) / L$

$\dot{x} = v \cdot \cos(\theta)$

$\dot{y} = v \cdot \sin(\theta)$

$\dot{\theta} = \omega$

where L = 0.052 m is the wheelbase. A PID controller was designed to steer the robot to each waypoint, taking the heading error as the control input. Tuning was done separately using a dedicated simulation script (pid_tuning.py) that compared an untuned (P-only), over-tuned (aggressive), and well-tuned setup. The final tuned parameters were **Kp = 2.5, Ki = 0.4, Kd = 0.6**.

**Integration.** The A* trajectory was exported to path.csv, which was imported into the Webots controller script for execution based on the tuned PID controller and odometric logging to actual_trajectory.csv.

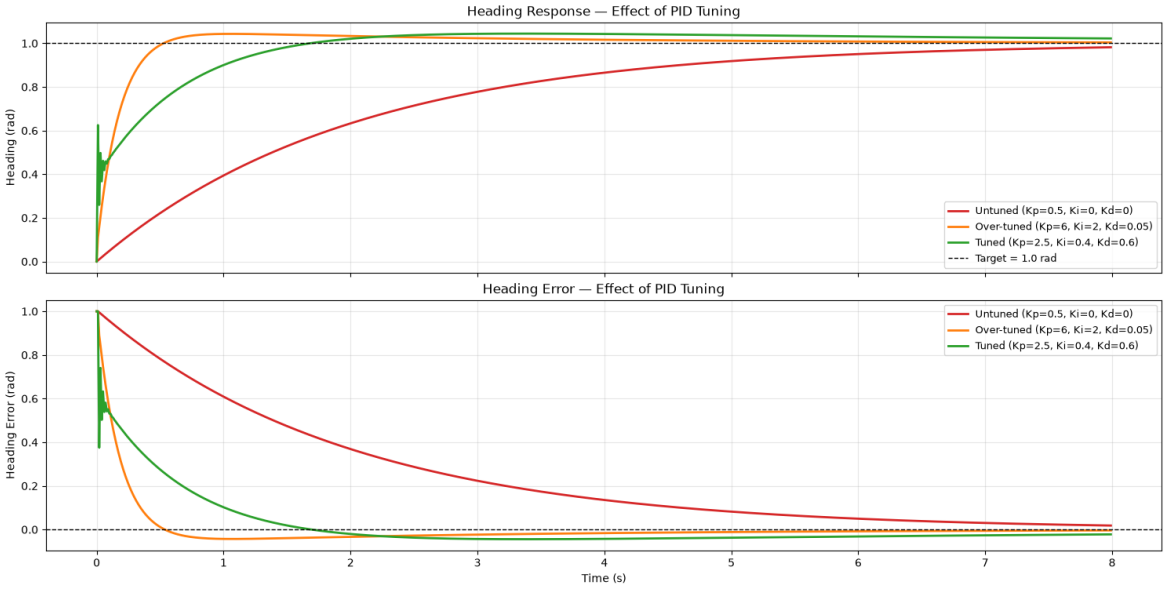



*Figure 2: Effect of PID tuning on heading response.*

## 2.2 Component B: Multi-Agent Swarm with Learning

**Simulation setup.** A 6×6 m workspace was built with three e-pucks placed at (-2.5, -2.5), (2.5, -2.5), and (0, 2.5), along with three target locations (represented by green cylinders) positioned at (-2, 1.5), (2, 1.5), and (0, -2). The world file used here is component_b.wbt; swarm_controller.py is used for all the agents.

**Agents' behavior and cooperative mission. The** mission is distributed coverage of an area: three agents have to reach individual targets, but before doing so, come to a common agreement on their meeting location by reaching consensus. There is no leader, and no centralized controller exists in the system.

**Communications Topology.** Communication via a file-based channel (the comms/ directory) acts as the simulation of a wireless broadcasting medium. Each agent saves its own vote into the file and reads the votes of the other two agents at each step. The file locking issue on Windows was addressed by retry loops and atomic operations (os.replace).

**Consensus algorithm.** Average consensus was implemented using:

$$x_i(k+1) = x_i(k) + \varepsilon \cdot \sum_{j \in \text{neighbours}(i)} (x_j(k) - x_i(k))$$

using step size $\varepsilon = 0.03$. The vote of each agent is a 2D point in the arena, and the team reaches the centre of mass of their initial votes.

**Particle swarm optimisation (PSO).** The task assignment problem – to which target will each agent go – was calculated offline using a Particle Swarm

Optimiser (pso_allocation.py), with 30 particles and 100 iterations. Every particle represents the permutation of targets indices; the fitness function is the total travelled distance. The solution was stored in assignment.csv and read by controllers.

**Machine learning.** A classifier based on a supervised feedforward neural network (MLPClassifier from scikit-learn) was trained to find the optimal assignment from the raw position data. The input variables for the network were the 3 × 3 distance matrix between the agents and targets (9 features), normalised with StandardScaler. The architecture had two hidden layers with 64 and 32 neurons and ReLU activations and early stopping applied. Ground truth was found by enumerating all possible 3! = 6 permutations.

# 3. Results and Evaluation

## 3.1 Component A Results

**Planner comparison.**

| Metric | Dijksta | A* |
|---|---|---|
| Path length (cells) | 56.57 | 56.57 |
| Nodes explored | 2135 | 41 |
| Time (ms) | 12.14 | 0.35 |

The two algorithms came up with identical path lengths, which shows that the two algorithms are theoretically optimal. The A* algorithm examined 52 times fewer nodes and was 34 times faster since it searches for the optimal solution by moving towards the goal.

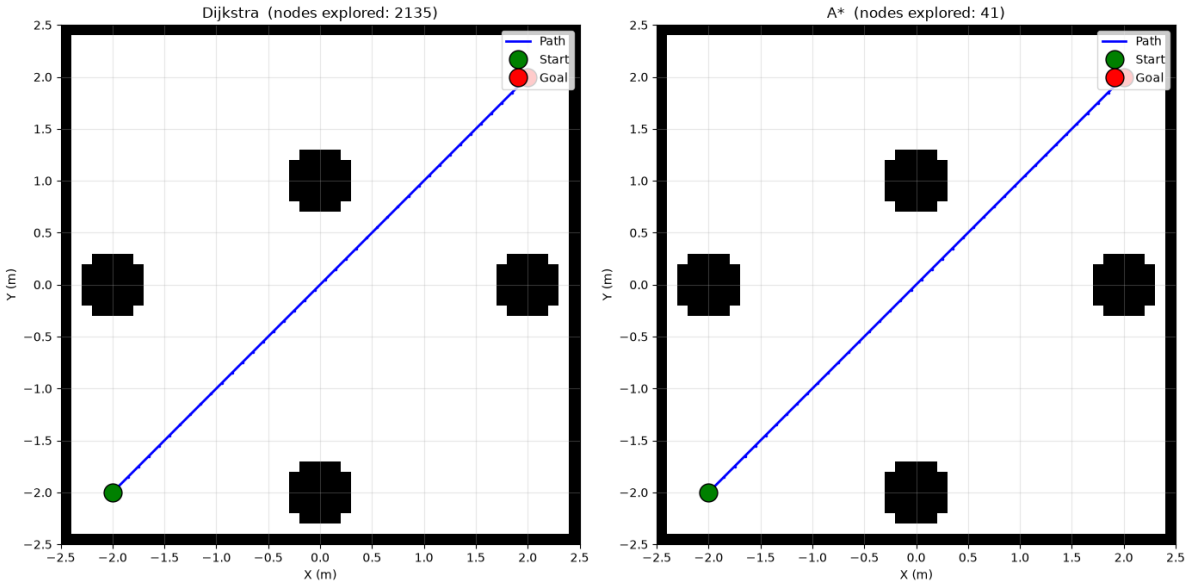



*Figure 3: Paths computed by Dijkstra and A**

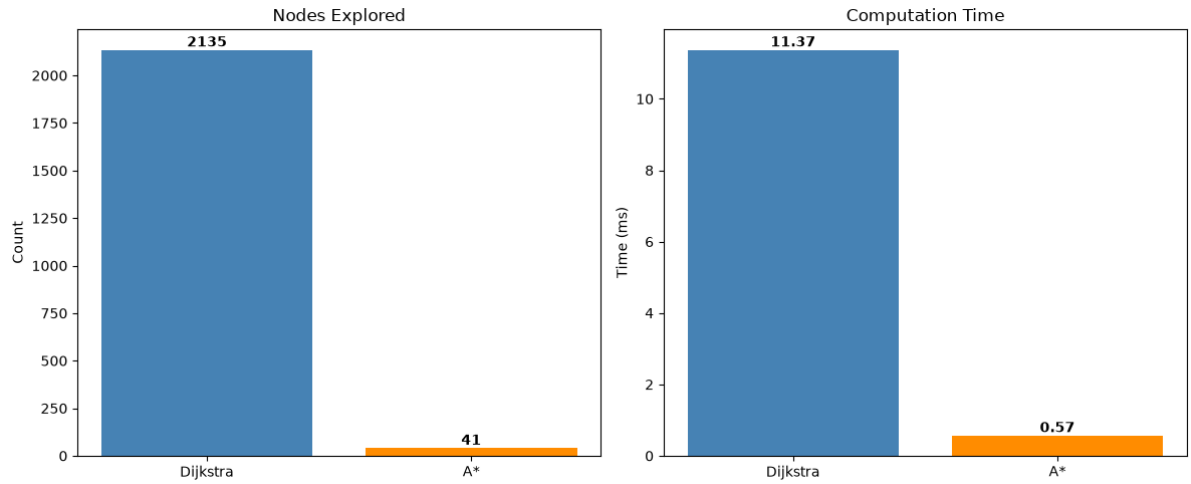



*Figure 4: Nodes explored and computation time*

**PID Tuning.** Three different values of the gains were considered: Untuned (Kp = 0.5), Over-tuned (Kp = 6, Ki = 2, Kd = 0.05), and tuned (Kp = 2.5, Ki = 0.4, Kd = 0.6). The untuned values of the gains caused slow response to change and steady-state error while the over-tuned values caused fast rise with oscillations and overshoots.

**Closed-loop trajectory tracking.**

| Metric | Value |
|---|---|
| Path length (planned) | 5.657 m |
| Path length (actual) | 5.615 m |
| Mean tracking error | 0.036 m |
| Max tracking error | 0.072 m |
| Time to goal | 57.6 s |

With a mean error of 3.6 cm, the robot followed its intended path. The largest error was 7.2 cm and was recorded when the robot was correcting its heading initially, thereafter PID functioned without any hitches.

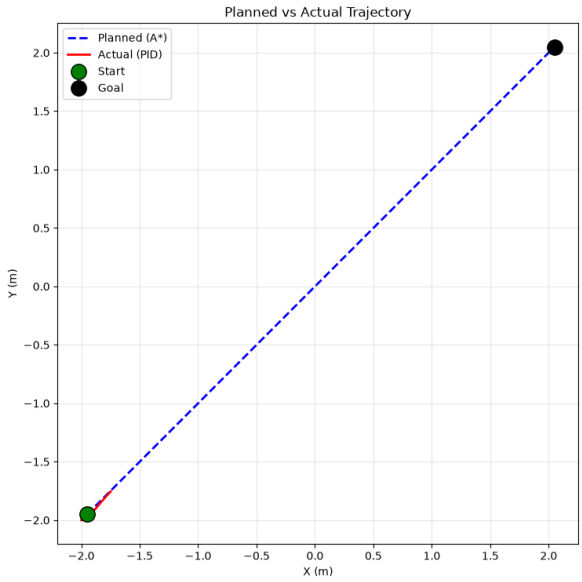



*Figure 5: Planned vs actual trajectory*

## 3.2 Component B Results

**Consensus convergence**. All three agents agreed on the same voting value (approximately (0, 0.67), which is the centroid of all three initial votes) in ~3 seconds of simulation time. The convergence followed an exponential pattern, as expected from the average consensus theory.

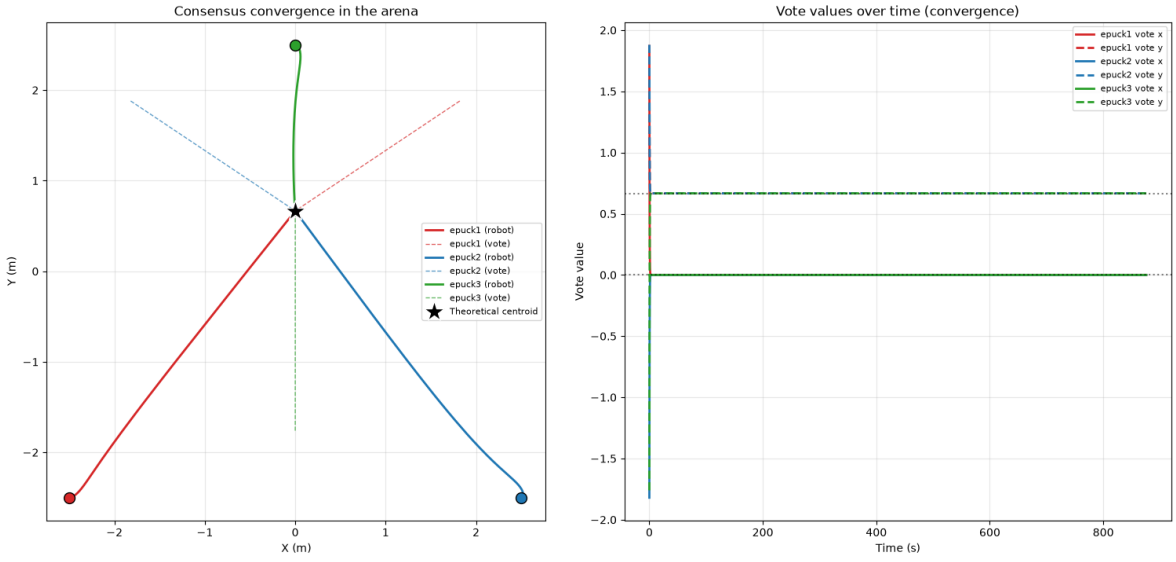



*Figure 6: Average consensus convergence*

**PSO Task Allocation.** The PSO-based solution algorithm produced the optimal assignment: epuck1 –> target3, epuck2 –> target2, epuck3 –> target1, with a total

travel distance of 8.82 meters. It should be noted that the optimal assignment produces a tie situation, i.e., two solutions (021 and 210) have the same cost, because of the symmetry of the arena about its origin point.

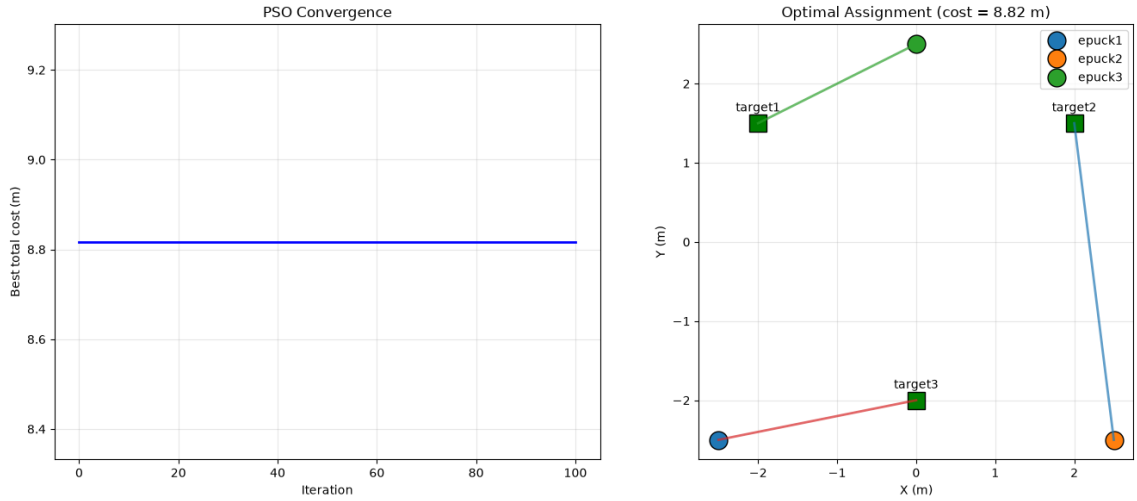



*Figure 7: PSO convergence and optimal assignment*

**Neural network performance.**

| Metric | Value |
|---|---|
| Test accuracy | 92.2% |
| Training converged | 63 iterations |
| Confusion matrix diagonal | Dominant |
| World config prediction | 210 (equally optimal to the brute-force answer) |

The MLP had an accuracy of 92.2% for testing on 1000 holdout samples. After analyzing the confusion matrix, it was found that the mistakes made by the model were largely due to the assignments having similar costs. In the particular setup of the world in the Webots simulation, two assignments (021 and 210) have similar cost value of 8.82 m. In this case, the classifier has chosen 210, which is one of the correct answers.

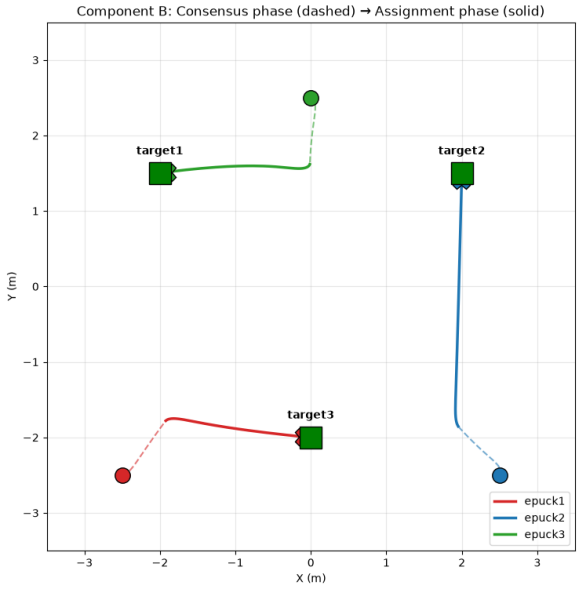



*Figure 8: Swarm behaviour across both phases*

**Centralised vs distributed decision-making.**

| Aspect | PSO (distributed) | Neural Network (centralised) |
|---|---|---|
| Optimality | Guaranteed | Approximate (92%) |
| Inference time | ~50 ms per solve | ~1 ms |
| Generalises to new configs | No | Yes |
| Requires training data | No | Yes |
| Scales to larger N | Yes | Retraining required |

PSO algorithm is better in situations where precision is needed and the search space is limited enough for exploration. Neural networks are better in terms of speed of inferring results, provided that the distribution of configurations is known, however, neural networks do not provide any guarantees about the results being correct.

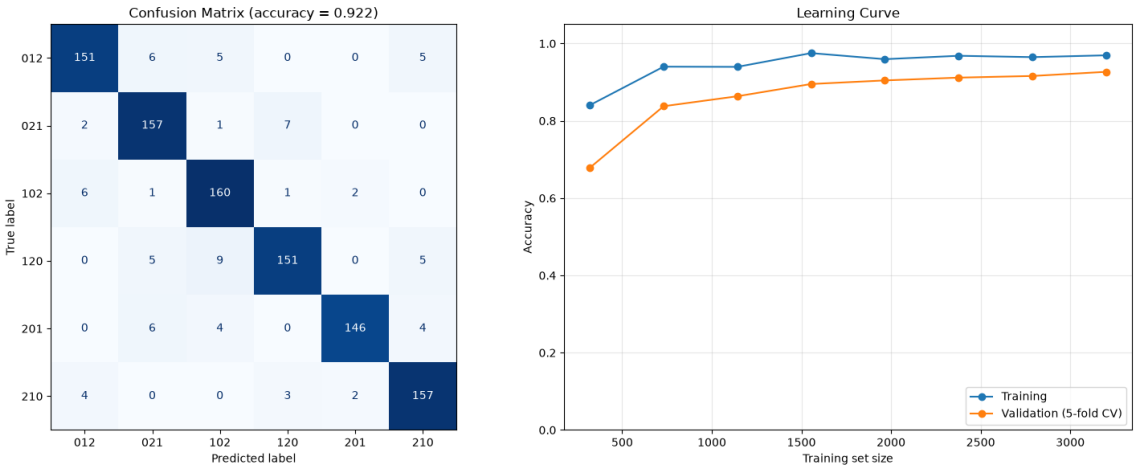



*Figure 9: Neural network performance.*

# 4. Ethics, Risk and Resposible Autonomy

## 4.1 Introduction

The creation of these two systems in the current project – a robot navigational pipeline that works alone and a swarm of agents with trained decision making – brings the ethical considerations behind autonomous systems much closer to concrete than a theoretical text could do. The fact is that each of our decisions during the construction of these systems, ranging from sensor fusion ratios to number of hidden layers in a neural network, implicitly embodies certain assumptions about the intended audience of the system, its capabilities, and consequences of its failure.

## 4.2 Bias and Reliability in Sensors, Simulation and Learned Models

Our perception pipelines incorporated the occupancy information provided by the LiDAR sensor into the camera imagery for constructing the environmental representation. Even in simulation, such a process is not free from bias. Specifically, the simulated LiDAR has an angular resolution and range limitation and discards thin or reflective surfaces. This is similar to what happens in real-world sensors. Our occupancy grid was constructed at a resolution of 0.1 m. The selection of the resolution is another bias: it is both sufficiently high to distinguish separate obstacles and sufficiently low for avoiding computationally expensive resolution.

The camera-based pipeline carries its own biases. First, the thresholding and contouring techniques work well only when there is no shadow, glare, or low contrast of obstacles relative to the floor. Our simulation does not take any of this into account.

A third component is incorporated into the learned model. It was recently discovered that even the most common AI models today are unfit for deployment on general physical robots because they failed important safety tests and even approved actions that could cause serious injury (EurekAlert, 2025). Although the neural network that we have used in this project is much more primitive than the language models which were assessed in that particular study, the idea remains the same – the model inherits all the properties of the synthetic dataset on which it was trained. We used uniform random locations in a 3x3 meter environment when training our MLP. Thus, the fact that it achieved 92% of accuracy is the measurement of its performance only on this kind of distribution, not the realistic ones of non-uniformly distributed elements in the warehouse or in a hospital corridor. In addition to that, we have noted that the ground truth had the tied assignments – assignments that have equal total cost. A supervised model can never learn to choose between them, no matter how much data it gets.

## 4.3 Transparency and Explainability

Our neural network is small, 64 and 32 hidden units, ReLU activations, but it is still a black box. We can give numbers for accuracy and confusion matrices, but we cannot point to a certain neuron and say that it is responsible for making the decision to assign the

configuration to assignment 210 rather than 021. This is fine for our experiment because both assignments were optimal, but it wouldn't be in a critical situation.

This is more important than it sounds. The inherent black box quality of deep neural networks and the associated inability to explain decisions pose serious problems to real-life applications, especially when operators have to be able to explain why something went wrong (ScienceDirect, 2025). If the warehouse robot delivers the parcel to the wrong worker because of the misclassification made by a neural network, the person responsible needs to know if the reason is bad sensor input, changed distribution of data during training, or ambiguous task definition. Otherwise, the problem will be impossible to solve.

A minor form of mitigation for use in this project is a back-up principle, whereby should the confidence of the neural network fall below a certain threshold, the decision would be deferred to the PSO solver, which would then be fully transparent and provable as far as its correctness goes in this particular case. The South African National Artificial Intelligence Policy Framework document, which came out in August 2024, lists transparency and explainability as a key pillar and requires that any decisions made using AI must be understandable to and interrogatable by those impacted (Bowmans Law, 2026).

### 4.4 Autonomy in Safety-Critical and Human-Shared Environments

The most challenging issue for us in this project lies in how the robot will behave when it meets a human. Our entire simulation does not involve any kind of human entity whatsoever, whether it be a pedestrian, an operator, or even just a bystander. While the e-pucks perform their pathfinding and stop at waypoints as required, there is no understanding of another agent who is not a robot. This is certainly a huge problem, as it represents a trend in autonomous systems research where safety is proved in already simplified scenarios.

There are three very clear and dangerous issues that appear in human-in-the-loop situations. First of all, the path planning algorithm assumes that everything else is static. Our implementation of the A* algorithm classifies each cell either as free or occupied, without any possibility of having a moving obstacle with unknown behavior. The second problematic element is the lack of an emergency stop function in our PID controller. If the robot meets a person in its path, nothing in the pipeline will tell it to stop, and the path will be considered valid by the planner. And lastly, the consensus protocol requires cooperative agents.

Solving these challenges would necessitate a multi-layered safety architecture, which would include a fast reactive layer such as the Braitenberg controller that is bundled in the Webots distribution along with the e-puck, and emergency stop initiated by specialized proximity sensors, in addition to a consensus algorithm capable of tolerating one malfunctioning node. The AI Policy Framework of South Africa stresses the notion of retaining the 'human in the loop' and mandates that no AI system can operate in isolation when making a decision with important implications for a person (Bowmans Law, 2026). However, none of the mentioned safety measures were relevant in our case scenario.

### 4.5 Fault Tolerance and Failure-Mode Mitigation in Multi-Agent Systems

The swarm experiment had its own type of concrete failure that arose from inter-agent communication based on files. Namely, writing a common file sometimes resulted in a PermissionError since two agents attempted to write a file at the same time. This was resolved via retries and atomic writes; however, this solution is unreliable in that it works simply because all three agents on the same computer resolve their timing issues in milliseconds. With wireless communication and real hardware with packet loss and asynchronous clocks, it would likely fail much more often.

In addition to that, the swarm does not have any way of handling failures. Should a controller of one of the agents crash during operation, then consensus algorithm would get stuck as other agents continue to read votes written before the crash and converge on a value with a phantom member of the swarm. There is no heartbeat, no timeouts, and no re-election process in place.

A body of research into Byzantine fault-tolerance of swarm robotics suggests that a single Byzantine robot, which does not act as expected because of failure or attack, can cause disruption of collaboration and coordination tasks such as mapping (Moroncelli et al., 2024). Multi-agent systems, resilient against Byzantine faults, usually include the following three components: redundancy (more agents than are needed for the task), timeout (agents that have not heard from another one in N seconds consider it out of commission), and reconfiguration (the surviving agents find an alternative plan of action). We have no such components, and admitting this is better than hiding behind pretenses.

### 4.6 Responsible Deployment in the South African Context

Implementing an autonomous system in South Africa involves issues that can be easily overlooked because the development process takes place in the simulation on a laptop in the lecture hall.

First of all, there is the issue of the Protection of Personal Information Act (Republic of South Africa, 2013), which will require more attention during any future implementation. According to Lexology, the Information Regulator of South Africa recently revealed that nearly 2,000 cases of data breach have been recorded since April 2025, which is an astonishing 40% increase in comparison with the previous year (Lexology, 2025). Our camera pipeline implemented in a public place would most likely identify faces and licence plates. The Processing of Personal Information according to POPIA needs a certain lawful basis and certain conditions.

The second one is infrastructure reliability. Unreliable infrastructure in South Africa is a basic impediment to the adoption of AI. Load shedding and connectivity problems result in practical questions regarding the ability of autonomous systems to operate without reliable electricity (Tandfonline, 2026). Multi-agent systems depending on a central server will cease operating during load-shedding. Our system of distributed consensus will be very effective in this situation, yet only provided that there is reliable communication – something that, as mentioned earlier, we do not have at this time.

The third one is the labour and inequality problem. Job replacement is the single biggest obstacle to acceptance of service robots in South Africa due to high unemployment rates in the country (Tandfonline, 2026). Warehousing, delivery and agricultural robots are actively tested in South Africa right now, and each of them replaces labour in sectors with high unemployment rates. This does not mean that deployment is a bad thing, yet it means that we should clearly understand who is gaining from it and who is losing. A robot that makes mining safer is not the same robot that replaces a shelf-stacker.

Finally, accountability. If an autonomous system causes harm in South Africa, it is not currently clear under existing law who is liable — the developer, the operator, the manufacturer, or the owner. South Africa does not yet have a dedicated AI Act; regulation is governed by existing legislation of general application, including POPIA, the Employment Equity Act, and the Labour Relations Act (Bowmans Law, 2026). The AI Policy Framework's pillar on Professional Responsibility aims to ensure that professionals deploying AI remain accountable for outcomes, but this is not yet binding legislation.

## 4.7 Conclusion

None of these issues are grounds for giving up the effort of building autonomous systems; rather, they are arguments in favor of being clear about what the capabilities and limitations of our systems are. The pipeline built in this project showcases real capability: perception, planning, control, coordination, and learning – but it also highlights how easy it is to use simulation to gloss over the difficult aspects of this challenge. The best thing this kind of system could do would be to show all of its assumptions explicitly, so that others can determine their validity.

## References


- Bowmans Law (2026) *South Africa: AI in the workplace – 1 of 6: The current regulatory landscape.* Available at: https://bowmanslaw.com/insights/south-africa-ai-in-the-workplace-1-of-6-the-current-regulatory-landscape/ [Accessed: 12 October 2026].
- EurekAlert (2025*) Popular AI models aren't ready to safely power robots.* Available at: https://www.eurekalert.org/news-releases/1105395 [Accessed: 12 October 2026].
- Lexology (2025) *South Africa: AI adoption by the SARB and FSCA — New insights, new risks, new rules.* Available at: https://www.lexology.com/library/detail.aspx?g=9d265dcc-894f-4704-8472-ee557936981a [Accessed: 12 October 2026].
- Moroncelli, A., Pacheco, A., Strobel, V., Lajoie, P.-Y., Dorigo, M. and Reina, A. (2024) *'Byzantine Fault Detection in Swarm-SLAM Using Blockchain and Geometric Constraints', Swarm Intelligence*. Available at: https://dl.acm.org/doi/10.1007/978-3-031-70932-6_4 [Accessed: 12 October 2026].
- Government Gazette (2013). *Government Gazette Republic of South Africa. [online]* Available at: https://www.gov.za/sites/default/files/gcis_document/201409/3706726-11act4of2013protectionofpersonalinforcorrect.pdf [Accessed 12 Oct. 2026].
- ScienceDirect (2025) *Ensuring trust in healthcare robotics: The essential role of explainable AI.* Available at: https://www.sciencedirect.com/science/chapter/edited-volume/abs/pii/B9780443239793000014 [Accessed: 12 October 2026].
- Tandfonline (2026) *Consumer Trust, Effort Expectancy Dimensions, and AI Service Robot Adoption: Refining the AIDUA Model in an Emerging Market.* Available at: https://www.tandfonline.com/doi/full/10.1080/08961530.2026.2656603 [Accessed: 12 October 2026].